electronics

IMPACT FACTOR 2.6

CITESCORE 5.3

Article

RETRACTED: Implementation of Binarized Neural Networks in All-Programmable System-on-Chip Platforms

Maoyang Xiang and Tee Hui Teo

Special Issue

Recent Advances in Microelectronics Devices and Integrated Circuit

Edited by

Dr. Teo Tee Hui and Dr. Akram Ben Ahmed

MDPI

https://doi.org/10.3390/electronics11040663

electronics

MDPI

Article

RETRACTED: Implementation of Binarized Neural Networks in All-Programmable System-on-Chip Platforms

Maoyang Xiang † and **Tee Hui Teo *,†**

Engineering Product Development, Singapore University of Technology and Design, 8 Somapah Road, Singapore 487372, Singapore; maoyang_xiang@mymail.sutd.edu.sg

* Correspondence: tthui@sutd.edu.sg

† These authors contributed equally to this work.

Abstract: The Binarized Neural Network (BNN) is a Convolutional Neural Network (CNN) consisting of binary weights and activation rather than real-value weights. Smaller models are used, allowing for inference effectively on mobile or embedded devices with limited power and computing capabilities. Nevertheless, binarization results in lower-entropy feature maps and gradient vanishing, which leads to a loss in accuracy compared to real-value networks. Previous research has addressed these issues with various approaches. However, those approaches significantly increase the algorithm's time and space complexity, which puts a heavy burden on those embedded devices. Therefore, a novel approach for BNN implementation on embedded systems with multi-scale BNN topology is proposed in this paper, from two optimization perspectives: hardware structure and BNN topology, that retains more low-level features throughout the feed-forward process with few operations. Experiments on the CIFAR-10 dataset indicate that the proposed method outperforms a number of current BNN designs in terms of efficiency and accuracy. Additionally, the proposed BNN was implemented on the All Programmable System on Chip (APSoC) with 4.4 W power consumption using the hardware accelerator.

Keywords: All Programmable System-on-Chip; Binarized Neural Networks; Convolutional Neural Network; Field-Programmable Gate Array

check for updates

Citation: Xiang, M.; Teo, T.H. RETRACTED: Implementation of Binarized Neural Networks in All-Programmable System-on-Chip Platforms. *Electronics* **2022**, *11*, 663. https://doi.org/10.3390/electronics11040663

Academic Editor: Spiros Nikolaidis

Received: 27 December 2021
Accepted: 14 February 2022
Published: 21 February 2022
Retracted: 15 January 2025

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The Binarized Neural Network (BNN) refers to the Convolutional Neural Network (CNN) which are made up of only bipolar data for weights and activation. When compared to a real-value CNN, it substitutes *XNOR* and Popcount for multiplication and accumulation within convolution operations, which economizes on memory and computing units and greatly facilitates the deployment of the model on resource-constrained devices [1]. Nonetheless, the accuracy of BNN model has always been substantially inferior to a real-value model with the same topology due to the limited information entropy.

Although recent studies such as MeliusNet [2], IRNet [3], and ReactNet [4] have worked hard to bring BNN' Top-1 on ImageNet dataset to more than 0.70, which is 3% lower than the corresponding real-value models with amounts of arithmetic operations. For the time being, it is difficult to achieve the balance between computational complexity and satisfying outcomes in sophisticated AI tasks.

On the other hand, BNN researches have made their way to optimizing the computing structure tailored to binary operations, which indicates platforms based on specific hardware to adapt to their forward propagation. Therefore, the programmable hardware is the appropriate candidate for BNN applications, especially Field-Programmable Gate Array (FPGA) and APSoC [5].

The FPGA platform can flexibly adjust its hardware units to different network structures [6–8] because of their customized circuit structure. Furthermore, given that embedded applications require the support of numerous peripherals and operating systems, the industry has developed the APSoC, which unites traditional Central Processing Unit (CPU)

RETRACTED

cores and FPGA units. It can make full use of the merits of the CPU and FPGA to deploy a variety of embedded applications.

Overall, the objective is to develop a highly effective BNN application with endeavours in network topologies and customized hardware architecture. Hence, a novel BNN topology was proposed and its FPGA-based acceleration cores, which were deployed on the APSoC device. The strategy incorporates two key ingredients:

1. The multi-scale BNN topology: Our proposal suggests a novel topology with few non-arithmetic operations that provides generalization ability of the network, which is based on the current BNN researches. It is well suited to programmable hardware due to its simple and regular architecture.
2. The BNN FPGA accelerator: An optimized FPGA accelerator design for multi-scale BNN topology, which were deployed to the Xilinx Zynq-7020 APSoC.

This paper explains the multi-scale BNN and the BNN FPGA accelerator with our design methodology in detail. Our customized topology and accelerator's performance are assessed and compared to other applications. In this section, the pros and cons of BNN applications and APSoC are illustrated. Section 2 provides an overview of the design methodology and specifications of the proposed topology and accelerator. Section 3 details the evaluation procedure and conducts a performance analysis on various tasks. Finally, Section 4 is summarized, along with a recommendation for future study directions.

2. Methods

The overview of the design methodology and specifications of the proposed topology and accelerator are presented in this section.

2.1. Design Methodology

Our approach to the BNN application concentrated on its performance in terms of accuracy and energy efficiency. Thus, an optimized typology and utilization of APSoC are critical for the approach which is shown in Figure 1. The detailed hardware implementation of the proposed BNN based on APSoC is described in Appendix A.

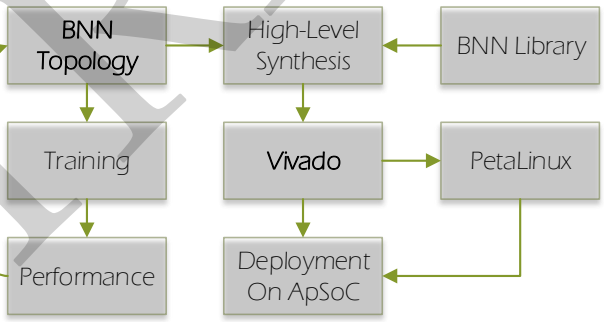

Figure 1. Flow chart illustrates the approach of our BNN development which involves procedures of training and deployment.

The first step is to design the typology, as it is the key factor to the performance. Given that training of BNN, similar to real-value network, relies on platforms with high computing capabilities, typologies are optimized on popular training platforms Pytorch and exported to network files manually. Then, the high-level synthesizer Vivado HLS 2019.1 compiles the network weight file and the BNN library into an intellectual property block that constitutes the hardware accelerator in Vivado developing kit. Following that, PetaLinux generates the Linux kernel and board support package for the accelerator and other peripherals. Finally, deployment also requires pre-processing and post-processing, which are handled by the processing system, as is shown in Figure 2.

RETRACTED

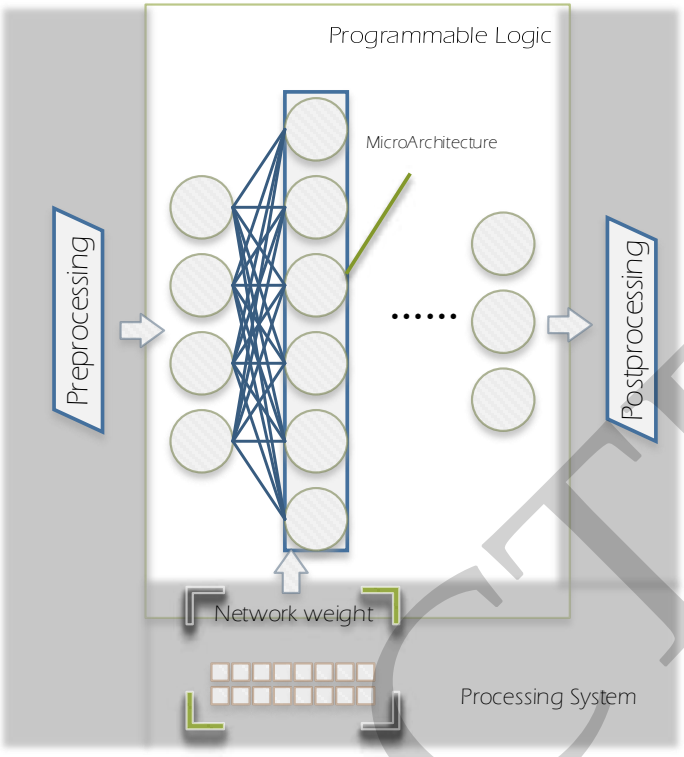

Figure 2. The programmable logic refers to programmable circuits and the processing system constitutes of one or more processing units which can not be customized.

The procedures of BNN application are split into two categories according to their characters. The processing system is in charge of procedures that do not require a large number of calculations, such as pre-processing, post-processing, and portions of forward propagation. Besides, the processing system initiates the accelerator by writing a weight file to programmable logic. While the processing logic deals with the vast majority of regular forward propagation.

For a starter, the dataset is required to be normalized within pre-processing procedure in an attempt to optimization of accuracy and converging speed. Given that it involves numbers of float-point operations, the processing system is in charge of normalization. Similarly, it sorts the confidence coefficient in the post-processing from the network with quick sort algorithm, a recursive algorithm, which could not deployed onto hardware resources. Hence, the programmable logic undertakes most of the high-density and regularly computing tasks and realizes the efficient use of hardware resources.

Irregular network typology, paradoxically, can significantly improve the performance of a network. However, it wastes large amounts of logic resources when it is deployed onto a hardware platform. Therefore, our approach takes both of them into consideration and designs the micro-architecture that is illustrated in the following paragraph. The micro-architecture was first proposed by Forrest et al. and indicates different higher-level building blocks which made up of numerous convolution layers that are organized in a specified way [9].

2.2. Binarized Neural Networks Analysis

When the BNN is resource-efficient, it should consume less memory with fewer operations, whether in forward or back-propagation, and have a consistent data structure to minimize data type conversion during operation. This section will examine the architectures of prior neural networks with excellent performance, including 1×1 convolution kernel, concatenation, short-cut connection, and batch-normalization.

RETRACTED

NIN [10] network was the first to suggest 1×1 convolution as a way to decrease the size of the feature map and the number of operations and parameters. It is the full connection of features in the same position. It has been widely utilized in practically all CNNs, such as GoogLeNet [11] and Residual Neural Network (ResNet) [12]. However, in BNN topology, the participation of 1×1 convolution will lead to a sharp deterioration in performance because the quantization process will lose much effective information.

Furthermore, concatenation is widely used in various topologies, because of its efficiency in enhancing network performance. Its goal is to improve the network's generalization by sparsifying via parallelizing numerous small convolution kernels. This technique achieves improved accuracy and a faster convergence speed with the same sized neural network.

Besides, the short-cut connection is an excellent way to boost network generalization ability. Its emergence overcomes the problem of the network's accuracy degrading as the depth rises, and then drastically increasing the network's depth. Due to the BNN's unique data structure, adding short-cuts to the topology is difficult unless a significant number of floating-point or integer addition operations are introduced. As a result, for fully BNN, a short-cut is not an optimal approach to improving generalization.

Finally, the last approach is batch-normalization which is a technique for unifying dispersed data and optimizing neural networks [13]. The primary effect of batch-normalization is to prevent internal covariate shift during neural network propagation and to keep the data features in the feature graph. Chen et al., on the other hand, argue that the its role is to smooth continuous optimization function [14].

The batch-normalization will involve the transformation of data types and the operation of floating-point arithmetic units. In order to further improve the processing speed on the hardware platform, in the forward propagation process of BNN, A *Sign* function with variable threshold value is adapted to replace batch-normalization and original *Sign* function where other divisors are regarded as constant, including means of features, variances and correction factors. Therefore, the activation process of BNN in forwarding propagation is:

$$Sign(x - \alpha) = \begin{cases} 1 & x \geq \alpha \\ -1 & x < \alpha \end{cases} \tag{1}$$

where:

$$\alpha = (-\frac{\beta\sqrt{\sigma^2 + \epsilon}}{\gamma} + \mu_0) \tag{2}$$

where μ_0 and σ^2 as the mini-batch mean and the mini-batch variance and γ, β are the learnable parameters.

2.3. Multi-Scale Binarized Neural Networks Topology

Finally, a complex BNN topology network is designed according to the optimization method described in the previous section, which is inspired by Res2Net [15]. The network structure consists of several MicroArchitectures, called binary inception modules, in series. Figure 3 shows the structure of the proposed binary inception module. The feature maps are split, input X_0^1, which channel is C, into two feature map subsets denoted by X_1^1 and X_1^2 separately. Thus, the channels of subsets are $\frac{C}{2}$, and the spatial size remains. Then X_1^2 convolves with the 3×3 convolution kernel, and output X_2^1 is concatenated to X_1^2, which convolves with the next 3×3 convolution kernel and so on. Finally, the module concatenates X_1^1, X_2^1, X_3^1, and X_4^1 as the output of the proposed module.

RETRACTED

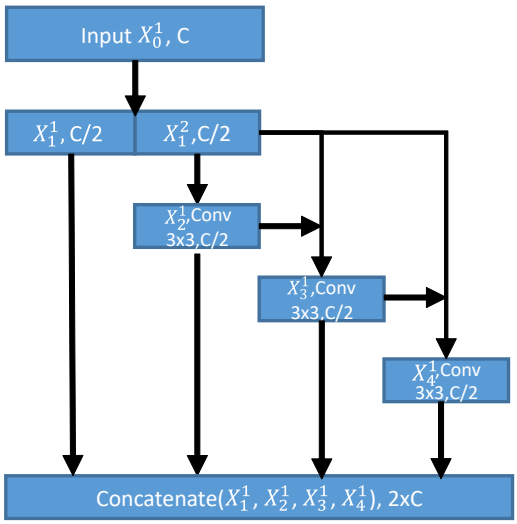

Figure 3. Binary inception module consists of multi-layers split and concatenation operations to enhance the generalization ability of the network.

Table 1 shows the proposed multi-scale BNN, which omits the activation layer and quantization for the briefing. The first layer is a unique layer since it takes $32 \times 32 \times 3$ float point number as input and generates $32 \times 32 \times 64$ input for the following layer. Then the binary inception module and max-pooling three times are applied until a $2 \times 2 \times 512$ feature is obtained, which will be unfolded to a one-dimensional vector. Finally, the vector connects to the confidence of 10 classification objects via two fully-connection layers. The architecture has 1,232,832 parameters which need 152.0 kB memory.

There is a very significant benefit of this design, that all binary convolution operations in the topology have similar characteristics, with size three kernels, one padding. Therefore, there are only few case statement and limited or constant for loops in our application, which greatly improves the reuse rate of our hardware.

Table 1. Proposed multi-scale BNN typology's layer structure and weight size.

Layer Name/Type	Output Size	Weight (1 bit)
Input Image	$32 \times 32 \times 3$	-
Conv1 (3×3, stride = 1)	$32 \times 32 \times 64$	1728
Binary Inception 1 + MaxPool	$16 \times 16 \times 128$	46,080
Binary Inception 2 + MaxPool	$8 \times 8 \times 256$	184,320
Binary Inception 3 + MaxPool	$4 \times 4 \times 512$	737,280
MaxPool	$2 \times 2 \times 512$	0
Binary Fully Connection	128	262,144
Binary Fully Connection	10	1280
Total		1,232,832

2.4. Binarized Neural Networks Field-Programmable Gate Array Accelerator

The hardware block diagram of the BNN FPGA accelerator core is shown in Figure 4 which is made up of four blocks: the feature map buffer, a Finite State Machine (FSM) controller, the expansion block, and an XNOR-CNT matrix. Each hardware block's design and optimization approach are described in detail in this section.

RETRACTED

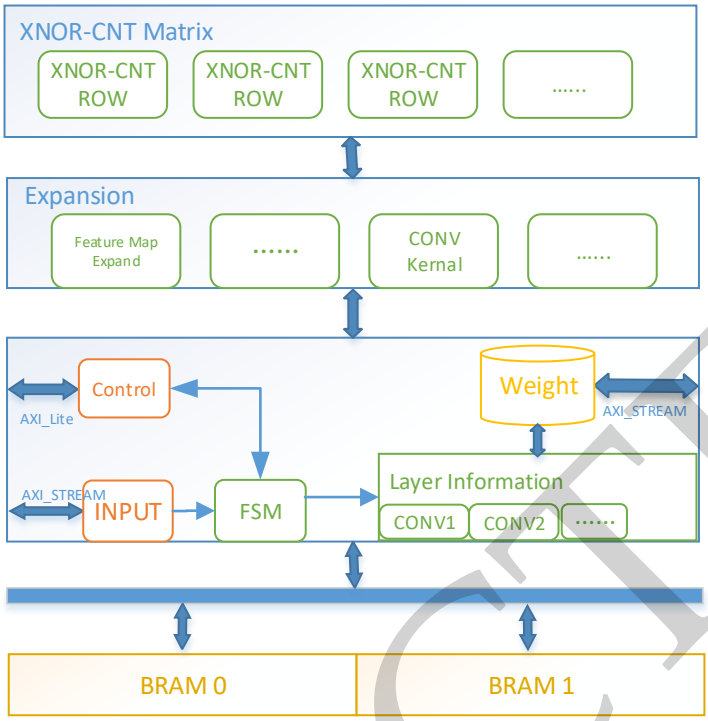

Figure 4. The BNN FPGA accelerator diagram is composed of four blocks: the feature map buffer, a FSM controller, the expansion block, and an XNOR-CNT matrix.

2.4.1. Finite State Machine Controller

The FSM controller is the center of the whole acceleration core, responsible for interface management, reading and writing of the feature map and weight, down-sampling the feature map, and holding the topology information for the entire neural network. It manages three interfaces: one AXI4-Lite interface for acceleration core control and two AXI4-STREAM interfaces for feature map and weight transmission. The AXI4-Lite interface can initiate, interrupt, and IDLE the acceleration core, as well as read its state, including standby, running, and completion.

The Expansion Block is directly connected to the FSM controller, which allows for the expansion of the feature map and convolution core into a particular vector for following operation. Simultaneously, it holds the results of the XNOR-CNT Matrix and arranges the output according to a specific data structure.

2.4.2. Expansion Block

The Expansion Block converts the feature map and convolution kernel to binary vectors. The convolution kernel remains during this operation, while the local feature map vector is iterated according to its location. The sliding step of the 3×3 convolution kernel is one unit in this project. The kernel scan feature map from left to right. When it moves one unit to the right, six of the nine feature vectors stay constant, but their storage location changes, while the other three feature vectors are replaced with new feature vectors from the feature map. When the convolution kernel realigns the first part of the row, it requires inputs three times to empty the data from the previous row, as shown in Figure 5. Thus, there is no need to supply extra bandwidth to transmit duplicate data throughout the sliding filtering process.

RETRACTED

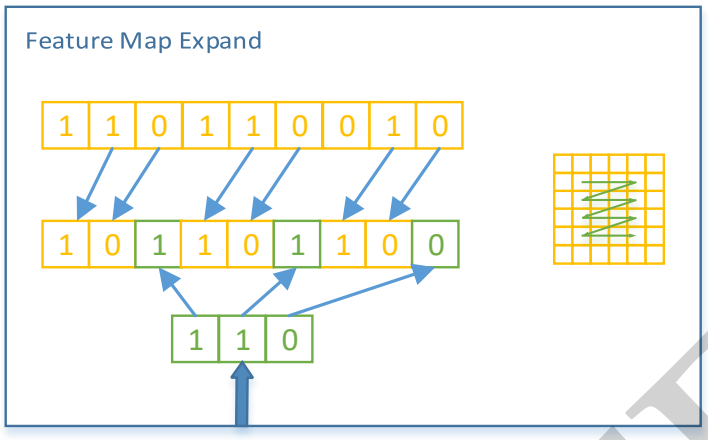

Figure 5. The Expansion Block emulates the sliding filtering process used by the small convolution kernel on the feature map.

2.4.3. XNOR-CNT Matrix

The XNOR-CNT Matrix is composed of *N* XNOR-CNT Rows, which are based on XNOR-CNT Core. XNOR-CNT Row is a fundamental binary convolutional unit that receives the feature vector, convolution kernel vector, activation threshold, and associated vector length from the Expansion Block, conducts an XNOR-popcount operation, and returns the activated results, as is shown in Figure 6. XNOR-CNT Core, as illustrated in the Figure 7, XNOR the input vectors first, and then *popcount* unit count the set-bits in the output vector and holds it in the accumulator. In order to match the current mainstream computing architecture, the width of the XNOR-CNT Core is designed to be 32 bits.

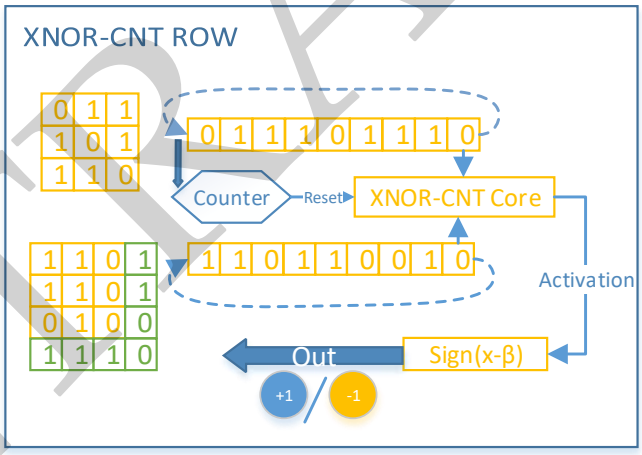

Figure 6. Hardware diagram of the XNOR-CNT ROW which conducts an XNOR-popcount operation.

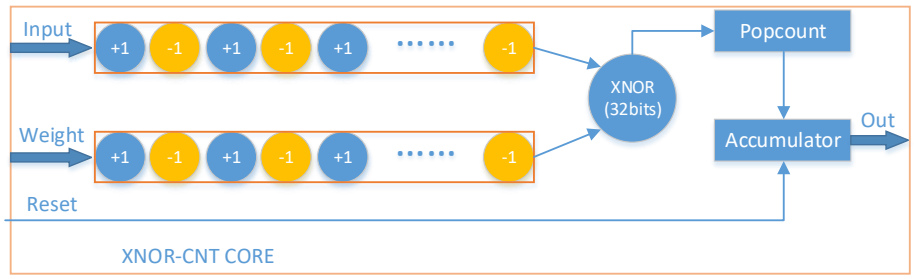

Figure 7. Hardware diagram of the XNOR-CNT core.

RETRACTED

2.4.4. Popcount Unit Block

The length of the output vector is denoted as L. The most straightforward approach is to visit each bit in the vector every clock cycle, which takes L clock cycles. Another approach is the Look-up table (LUT), which creates the mapping between the vector and the *popcount* result and takes only one clock cycle but consumes 2^L space to store the LUT. The last approach is the dichotomy algorithm which count the number of set bits within neighboring groups. It groups two neighboring bits and counts the number of set bits inside each group by summation. Then, two neighboring results are summed up, and so on, as indicated in Figure 8.

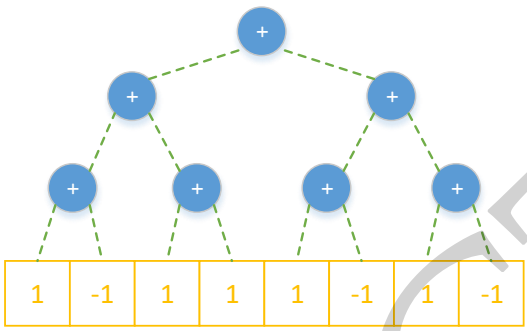

Figure 8. Popcount dichotomy algorithm.

This approach can count the bits in a feature vector of length L within a $log_2 L$ clock cycle. This structure, however, suffers from redundancy in hardware space since it requires $\frac{(1+log_2 L)\times log_2 L}{2}$ adders. To further minimize hardware resource usage, a generic hardware structure *popcount* unit has been developed for the computing process, which is comprised of four 8-bit full adders. It takes a 32-bit feature vector α, a bit-shifting unit s, and a bit-mask m as input and returns $Popcount_Unit(\mathbf{ff},\ s,\ m) = (\mathbf{ff} \cap m) + ((\mathbf{ff} >> s) \cap m)$. It uses the combined operation of mask and shifting to realize the summation of adjacent bits or bits group. Hence, the reuse of adders is realized to reduce the consumption of logic resources. Finally, the algorithm for *popcount* is shown in Algorithm 1.

Algorithm 1: Popcount using algorithm

Input: a 32 bits binary vector α
$\alpha = Popcount_Unit(\alpha, 1, \texttt{0x55555555})$ `/* Two bits in a Group */`
$\alpha = Popcount_Unit(\alpha, 2, \texttt{0x33333333})$ `/* Four bits in a Group */`
$\alpha = Popcount_Unit(\alpha, 4, \texttt{0x0F0F0F0F})$ `/* Eight bits in a Group */`
Result: $(\alpha \times \texttt{0x01010101}) >> 24$

2.4.5. Parallelization

The previous section described the specific structure of binary convolution hardware in detail, and this part mainly focuses on how to plan these hardware structures to speed up network inference. The most direct way is to parallelize the convolutional operations by duplicating multiple sets of the same computing units. The convolution of the feature maps is a high-dimensional operation process, so the selection of parallel dimensions directly affects the inference efficiency of the network. XNOR-CNT Matrix parallelizes from two aspects: the input channel and the output channel, because the length and width of the convolution kernel feature graph are irregular, odd numbers in most cases. The number of input and output channels can be artificially designed to make better use of existing hardware resources. So, in the end, the convolution algorithm of BNN is designed as follows (Algorithm 2).

RETRACTED

Algorithm 2: Binary Convolution Algorithm

Input: The weight vector, $\mathbf{F}_{z,k,k,\frac{c_0}{32}}$, for output channel z is composed of 32 elements on the input dimension c_0 of four-dimensional weight W_{c_1,k,k,c_0};
Input: The feature vector, $\mathbf{I}_{h_i,w_i,\frac{c_0}{32}}$, is composed of 32 elements on the input dimension c_0 of input feature map $\mathbf{I}_{h_i,w_i,c_0}$;
Input: The length, width and channels of the input and output feature map **I**,**O** are h_i, w_i, c_0 and h_o, w_o, c_1, respectively and $h_i = h_o, w_i = w_o$;
Input: Variable threshold for C_1 channels' output map $\mathbf{ff}_{C_1}$;
Result: Output feature map **O**

Loading $\mathbf{F}_{z,k,k,\frac{c_0}{32}}$;
Loading $\mathbf{I}_{h_i,w_i,\frac{c_0}{32}}$;
Loading α_{C_1};
for $m_2 = 1$ *to* $\frac{c_1}{N}$ **do**
  for $m_1 = 1$ *to* N **do**
    /* Loop will unrolled to N parallel architectures */
    for $j_1 = 1$ *to* h_i **do**
      /* Loop for the feature map */
      for $i_1 = 1$ *to* w_i **do**
        for $j = \frac{k}{2}$ *to* $-\frac{k}{2}$ **do**
          /* Loop for the convolution kernel */
          for $i = \frac{k}{2}$ *to* $-\frac{k}{2}$ **do**
            for $m = 1$ *to* $\frac{c_0}{32}$ **do**
              $a += PC\,(\mathbf{F}_{z,j,i,m} \otimes \mathbf{I}_{j_1+j,i_1+i,m})$ /* PC represents popcount algorithm */
            end
          end
        end
        $\mathbf{O}_{(m_2 \times N + m1, j_1, i_1)} = Sign(a - \mathbf{ff}_{m_2 \times N + m1})$
      end
    end
  end
end

3. Results

The last two sections outline the multi-scale BNN and the BNN FPGA Accelerator. This section conducts an in-depth analysis of this system from a variety of perspectives. First, the multi-scale BNN's performance was assessed and compared with prior studies in Section 3.1. Following that, the BNN FPGA accelerator's performance will be analyzed and some enhancements based on various BNN topologies are proposed in Section 3.2.

3.1. Multi-Scale Binarized Neural Networks Performance

This section examines the accuracy and algorithm complexity of multi-scale BNN by comparing the top results of current BNN networks. There are some limits to measuring the efficiency of our topology since BNNs performance will fluctuate with many factors, such as data enhancement, approaches of the optimizer, and so on. BNN topology comparison in this section is limited to its scenario and does not reveal the full potential of this type of BNN structure.

3.1.1. Accuracy

Because of the limited semantic expression capabilities of the BNNs and the limited memory footprint of the embedded platform, the dataset of tiny images, CIFAR-10, was used as the test benchmark. In Table 2, Bit-Width (W/A) represents the width of the weight

RETRACTED

and the activated data, respectively. Both the width of weight and the data after activation are one in the fully BNN. On the contrary, if the width of the activated data is greater than one, it is a partial BNN.

Table 2. Image classification performance of binary neural networks on CIFAR-10 dataset [16].

Method	Bit-Width (W/A)	Topology	Acc. (%)
BinaryConnect [17]	1/32	VGG-Small	91.7
BNN [18]	1/1	VGG-Small	89.9
XNOR-Net [19]	1/1	VGG-Small	89.8
LQ-Nets[20]	1/32	ResNet-20	90.1
BBG [21]	1/1	ResNet-20	85.3
BCGD [22]	1/4	VGG-11	89.6
IR-Net [23]	1/1	VGG-Small	90.4
CI-BCNN [24]	1/1	VGG-Small	92.5
Multi-scale BNN(Ours)	1/1	Customized	91.5

As a fully BNN topology, multi-scale BNN achieves 91.5% accurate at recognizing, which is close to the best results achieved by BNN in CIFAR-10 testing, 91.7% (BinaryConnect) and 92.5% (CI-BCNN). Nevertheless, there are many non-bitwise operations involved in the inference process of these two structures. Therefore, the proposed multi-scale BNN is an efficient BNN topology.

3.1.2. Algorithm Complexity

The multi-scale BNN requires three operations, XNOR-Popcount, pooling, and concatenation. There are four pooling layers in our BNN which requires 29,696 comparison operations. Meanwhile, XNOR-Popcount is the essence of the forward propagation process, which requires a total of 32,104,960 operations, which is 57% less than that of the classic BNN [18]. In terms of memory requirements, Table 1 shows the number of parameters required for each layer of BNN in multi-scale BNN, 154KB in total.

3.2. Binarized Neural Networks Field-Programmable Gate Array Accelerator Performance

To evaluate the efficiency of our accelerator's approach for topologies, SVHN, and CIFAR-10 were selected to test the proposed approach. The SVHN comprises of the numbers' images on the street, while the CIFAR-10 has ten classes of pictures, each with six thousand photos. The test was conducted at 650 MHz CPU frequency, 1050 MHz DRAM frequency and 142.85 MHz clock frequency for the accelerator. Linux kernel was generated by the PetaLiunx 2019.1 and kernel version was 4.19 LTS.

The results of SVHN test is shown in Table 3. Our self-defined accelerated kernel achieves the highest accuracy of 97.0%, which is the same as ReBNet [25], with a minimum of LUT and Block Random-access Memory (BRAM) resources, although efficiency is 1561 FPS/W, lower than the results of FINN [26] and FBNA [27].

Table 3. Deployment performance of BNNs on SVHN [28].

Method	FINN [26]	FBNA [27]	ReBNet [25]	Ours
Acc.	94.9	96.9	97.0	97.0
Topology	CNV-6	CNV-6	CNV-6	Customized
Platform	Zynq-7045	Zynq-7020	Zynq-7020	Zynq-7020
LUTs	46,253	29,600	53,200	27,342
BRAMs	186	103	280	94
FPS	21,900	6451	100	5310
Power (W)	11.7	3.2	-	3.5
Effi. (FPS/W)	1871	2051	-	1517

RETRACTED

The CIFAR-10 test results is shown in Table 4. Our self-defined acceleration kernel achieves the highest accuracy in CIFAR-10, 91.3%, although its efficiency is 122 FPS/W close to FINN [29]. From the above tests, the proposed multi-scale BNN has significant advantages for complex semantic scenarios, and the networks can achieve high accuracy while maintaining high efficiency. At the same time, the proposed BNN FPGA accelerators can adapt to the complex BNN topology very well.

Table 4. Deployment performance of BNNs on CIFAR-10 [16].

Method	Acc.	Topology	Platform	LUTs	BRAMs	FPS	Power (W)	Effi. (FPS/W)
Zhou et al. [30]	66.6	CNV-2	Zynq-7045	20,264	-	-	-	-
FINN-R [31]	80.1	CNV-6	Zynq-7020	25,700	242	-	2.3	-
FINN [26]	80.1	CNV-6	Zynq-7045	46,253	186	21,900	11.7	819
FINN [29]	80.1	CNV-6	Zynq-7020	42,853	270	445	2.5	178
Nakahara [32]	81.8	CNV-6	Zynq-7020	14,509	32	420	2.3	182
Ours	91.5	Customized	Zynq-7020	37,286	130	537	4.4	122

4. Conclusions

The proposed BNN develops a fully functional proof-of-concept system using the Xilinx Zynq-7000 programmable platform. This work demonstrates the feasibility of implementing integrated BNN using APSoC in FPGA. Although the system performs well, there are room of improvements in throughput and energy efficiency. The stringent requirements for embedded BNN in terms of platform size, efficiency, and computing power remains as challenges.

Artificial Intelligence (AI) is a promising technology, but the development is still hazy due to the lack of a comprehensive set of theories to support the relevant designs [33,34]. Is deep neural networks or reinforcement learning the future of AI? Additionally, there is uncertainty around the growth of computer platforms since Moore's Law is progressively eroding and AI is heavily reliant on computational resources [35]. Future AI applications on edge devices may be cloud-based or operate on local devices at a modest scale. In the near future, there will be several computer platforms, each with its own set of benefits.

Author Contributions: Conceptualization, M.X. and T.H.T.; methodology, M.X. and T.H.T.; software, M.X. and T.H.T.; validation, M.X. and T.H.T.; formal analysis, M.X. and T.H.T.; investigation, M.X. and T.H.T.; resources, T.H.T.; data curation, M.X. and T.H.T.; writing—original draft preparation, M.X. and T.H.T.; writing—review and editing, M.X.; visualization, M.X. and T.H.T.; supervision, M.X.; project administration, M.X.; funding acquisition, M.X. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The evaluation for the BNN was based on two database, SVHN and CIFAR-10. Please refer to SVHN testset at http://ufldl.stanford.edu/housenumbers (accessed on 18 December 2021) and CIFAR-10 testset at http://www.cs.toronto.edu/~kriz/cifar.html (accessed on 18 December 2021) , respectively.

Acknowledgments: The authors would like to thank Wong Thin Sek for his supports to this project.

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
APSoC	All Programmable System-on-Chip
AXI	Advanced eXtensible Interface
BNN	Binarized Neural Networks
BRAM	Block Random-access Memory

RETRACTED

CNN	Convolutional Neural Network
CPU	Central Processing Unit
DSP	Digital Signal Processing
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
LUT	Look-up table
PS	Processing System
ResNet	Residual Neural Network
SoC	System-on-Chip

Appendix A

Due to the low power consumption and full programmability of the ZYNQ-7020 SoC, this project utilizes it as a low power development platform. The development platform is primarily comprised of a ZYNQ XC7Z020-1CLG400C, an SD card, an Ethernet interface, and 1 GB DDR3 memory, as is shown in Figure A1. The ZYNQ XC7Z020-1CLG400C is based on the Xilinx SoC architecture, which combines dual-core or single-core processing systems based on the ARM Cortex® processor-A9 and 28 nm Xilinx programmable logic, Artix-7 [36], in a single chip. The ZYNQ-7020's Artix-7 FPGA fabric has 85k logic cells, 53,200 LUTs, 4.9 MB (140 blocks) of BRAM, and 220 DSP slices.

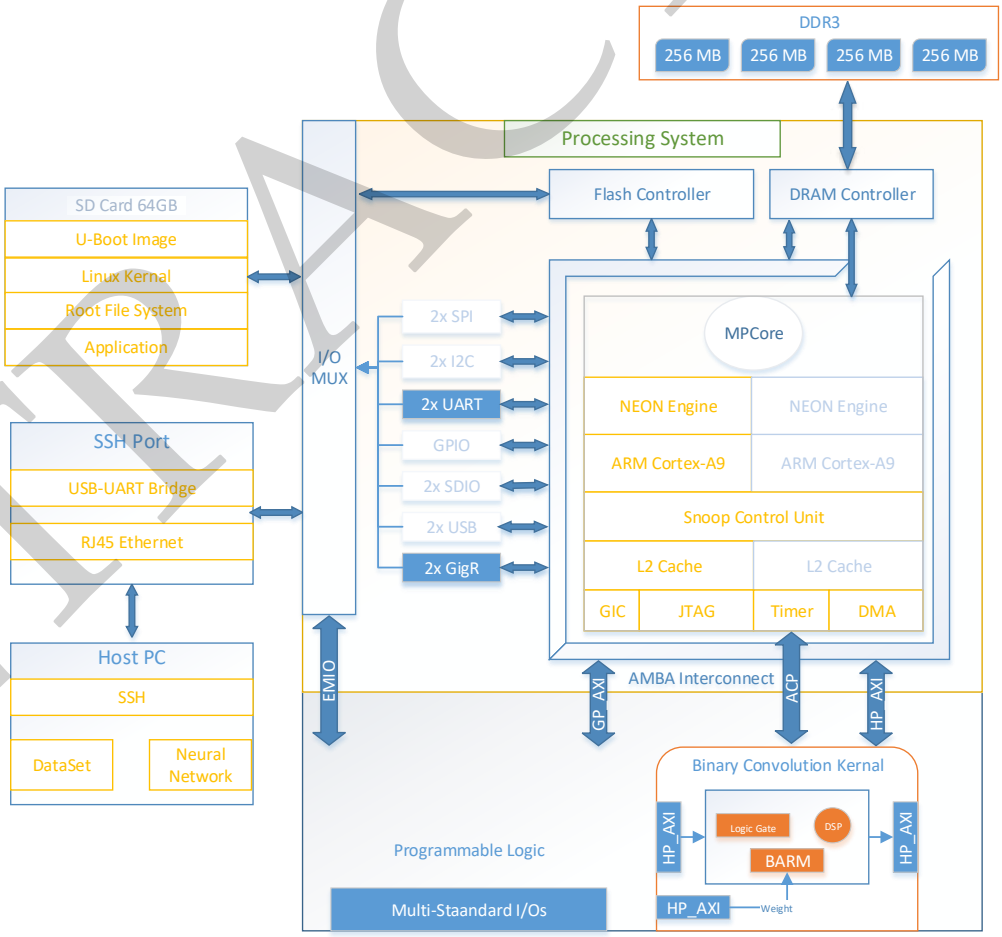

Figure A1. The hardware block diagram of the BNN implementation based on APSoC.

In processing system, 1 GB DDR3 memory supports the operating system, whereas the hardware acceleration core in PL may also read and write memory through DMA since there is no independent DDR3 memory for the PL. The boot mode is set to SD card. The SD card contains the U-Boot and Linux Kernel. The chip will first execute U-boot in order to finish the system's startup and basic checks and then load the Linux Kernel 4.19 LTS and

RETRACTED

Root File System. The Linux system activates only one of the operating cores, a serial port (for Linux Kernal Terminal) and an Ethernet. The host then establishes an SSH connection to the development platform via Ethernet and downloads the appropriate applications and test data sets to the development platform.

The CPU and DDR3 frequency are set to 650 MHz and 525 MHz, respectively. A 142.85 MHz clock drives the hardware accelerator.

References

1. Ehliar, A. Area efficient floating-point adder and multiplier with IEEE-754 compatible semantics. In Proceedings of the 2014 IEEE International Conference on Field-Programmable Technology (FPT), Shanghai, China, 10–12 December 2014; pp. 131–138.
2. Bethge, J.; Bartz, C.; Yang, H.; Chen, Y.; Meinel, C. MeliusNet: Can binary neural networks achieve mobilenet-level accuracy? *arXiv* **2020**, arXiv:2001.05936.
3. Qin, H.; Gong, R.; Liu, X.; Shen, M.; Wei, Z.; Yu, F.; Song, J. Forward and backward information retention for accurate binary neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020; pp. 2250–2259.
4. Liu, Z.; Shen, Z.; Savvides, M.; Cheng, K.T. Reactnet: Towards precise binary neural network with generalized activation functions. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2020; pp. 143–159.
5. Xiang, M.; Teo, T.H. A Multi-scale Binarized Neural Network Application based on All Programmable System on Chip. In Proceedings of the 2021 IEEE 14th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC), Singapore, 20–23 December 2021; pp. 151–156.
6. Zhang, C.; Li, P.; Sun, G.; Guan, Y.; Xiao, B.; Cong, J. Optimizing fpga-based accelerator design for deep convolutional neural networks. In Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, Monterey, CA, USA, 22–24 February 2015; pp. 161–170.
7. Li, Y.; Liu, Z.; Xu, K.; Yu, H.; Ren, F. A 7.663-TOPS 8.2-W energy-efficient FPGA accelerator for binary convolutional neural networks. *FPGA* **2017**, 290–291.
8. Omondi, A.R.; Rajapakse, J.C. *FPGA Implementations of Neural Networks*; Springer: Berlin/Heidelberg, Germany, 2006; Volume 365.
9. Iandola, F.N.; Han, S.; Moskewicz, M.W.; Ashraf, K.; Dally, W.J.; Keutzer, K. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5 MB model size. *arXiv* **2016**, arXiv:1602.07360.
10. Lin, M.; Chen, Q.; Yan, S. Network in network. *arXiv* **2013**, arXiv:1312.4400.
11. Szegedy, C.; Liu, W.; Jia, Y.; Sermanet, P.; Reed, S.; Anguelov, D.; Erhan, D.; Vanhoucke, V.; Rabinovich, A. Going deeper with convolutions. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Boston, MA, USA, 7–12 June 2015; pp. 1–9.
12. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778.
13. Ioffe, S.; Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning*; PMLR: London, UK, 2015; pp. 448–456.
14. Chen, L.; Fei, H.; Xiao, Y.; He, J.; Li, H. Why batch normalization works? A buckling perspective. In Proceedings of the 2017 IEEE International Conference on Information and Automation (ICIA), Macau, China, 18–20 July 2017; pp. 1184–1189.
15. Gao, S.; Cheng, M.M.; Zhao, K.; Zhang, X.Y.; Yang, M.H.; Torr, P.H. Res2net: A new multi-scale backbone architecture. *IEEE Trans. Pattern Anal. Mach. Intell.* **2019**, *43*, 652–662. [CrossRef] [PubMed]
16. Krizhevsky, A.; Hinton, G. *Learning Multiple Layers of Features from Tiny Images*; Technical Report; University of Toronto: Toronto, ON, Canada, 2009.
17. Courbariaux, M.; Bengio, Y.; David, J.P. Binaryconnect: Training deep neural networks with binary weights during propagations. *arXiv* **2015**, arXiv:1511.00363.
18. Courbariaux, M.; Hubara, I.; Soudry, D.; El-Yaniv, R.; Bengio, Y. Binarized Neural Networks: Training deep neural networks with weights and activations constrained to +1 or −1. *arXiv* **2016**, arXiv:1602.02830.
19. Rastegari, M.; Ordonez, V.; Redmon, J.; Farhadi, A. Xnor-net: Imagenet classification using binary convolutional neural networks. In *European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2016; pp. 525–542.
20. Zhang, D.; Yang, J.; Ye, D.; Hua, G. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In Proceedings of the European Conference on Computer Vision (ECCV), Munich, Germany, 8–14 September 2018; pp. 365–382.
21. Shen, M.; Liu, X.; Gong, R.; Han, K. Balanced binary neural networks with gated residual. In Proceedings of the ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Barcelona, Spain, 4–8 May 2020; pp. 4197–4201.
22. Xu, Y.; Dong, X.; Li, Y.; Su, H. A main/subsidiary network framework for simplifying binary neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 7154–7162.
23. Gong, R.; Liu, X.; Jiang, S.; Li, T.; Hu, P.; Lin, J.; Yu, F.; Yan, J. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In Proceedings of the IEEE/CVF International Conference on Computer Vision, Seoul, Korea, 27–28 October 2019; pp. 4852–4861.

RETRACTED

24. Wang, Z.; Lu, J.; Tao, C.; Zhou, J.; Tian, Q. Learning channel-wise interactions for binary convolutional neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Long Beach, CA, USA, 15–20 June 2019; pp. 568–577.
25. Ghasemzadeh, M.; Samragh, M.; Koushanfar, F. ReBNet: Residual Binarized Neural Networks. In Proceedings of the 2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Boulder, CO, USA, 29 April–1 May 2018; pp. 57–64.
26. Umuroglu, Y.; Fraser, N.J.; Gambardella, G.; Blott, M.; Leong, P.; Jahre, M.; Vissers, K. Finn: A framework for fast, scalable Binarized Neural Networks inference. In Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays, Monterey, CA, USA, 22–24 February 2017; pp. 65–74.
27. Guo, P.; Ma, H.; Chen, R.; Li, P.; Xie, S.; Wang, D. FBNA: A fully Binarized Neural Network Accelerator. In Proceedings of the 2018 28th IEEE International Conference on Field Programmable Logic and Applications (FPL), Dublin, Ireland, 26–30 August 2018; pp. 51–513.
28. Netzer, Y.; Wang, T.; Coates, A.; Bissacco, A.; Wu, B.; Ng, A.Y. Reading Digits in Natural Images with Unsupervised Feature Learning. In Proceedings of the NIPS Workshop on Deep Learning and Unsupervised Feature Learning, New York City, NY, USA, 12–15 December 2011.
29. LeCun, Y.; Bottou, L.; Bengio, Y.; Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **1998**, *86*, 2278–2324. [CrossRef]
30. Zhou, Y.; Redkar, S.; Huang, X. Deep learning binary neural network on an FPGA. In Proceedings of the 2017 IEEE 60th International Midwest Symposium on Circuits and Systems (MWSCAS), Boston, MA, USA, 6–9 August 2017; pp. 281–284.
31. Blott, M.; Preußer, T.B.; Fraser, N.J.; Gambardella, G.; O'brien, K.; Umuroglu, Y.; Leeser, M.; Vissers, K. FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks. *ACM Trans. Reconfig. Technol. Syst. (TRETS)* **2018**, *11*, 1–23. [CrossRef]
32. Nakahara, H.; Fujii, T.; Sato, S. A fully connected layer elimination for a binarized convolutional neural network on an FPGA. In Proceedings of the 2017 27th IEEE International Conference on Field Programmable Logic and Applications (FPL), Ghent, Belgium, 4–8 September 2017; pp. 1–4.
33. Teo, T.H.; Yi Shu, T. Fast Object Detection on the Road. In Proceedings of the 2020 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), Ha Long, Vietnam, 8–10 December 2020; pp. 173–176.
34. Teo, T.H.; Yi Shu, T.; Wei Ming, T. Deep-Learning Learning by Design. In Proceedings of the 10th IEEE Annual International Conference on Computer Science Education: Innovation and Technology (CSEIT), Bangkok, Thailand, 1 August 2019; pp. 87–90.
35. Teo, T.H.; Yi Su, T.; Wei Ming, T. Tumour Detection using Convolutional Neural Network on a Lightweight Multi-Core Device. In Proceedings of the 2019 IEEE 13th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC), Singapore, 1–4 October 2019; pp. 87–92.
36. Przybus, B. Xilinx redefines power, performance, and design productivity with three new 28 nm fpga families: Virtex-7, kintex-7, and artix-7 devices. *Xilinx White Paper.* **2012**, *WP373*, 1–10.

RETRACTED